

C# OOP Assignment 06 Solution

Student Name: Ahmed Mohamed Saeed Nofal

Group Code: ONL5_SWD5_S4

Course: Object-Oriented Programming in C#

Submission Date: September 2026

Assignment Requirements Summary

In this assignment, we need to design and implement an employee management structure in C# using OOP principles:

- **HiringDate Class:** Contains fields for Day, Month, and Year with full validation.
- **Gender Enum:** Restricts employee gender strictly to M (Male) or F (Female).
- **SecurityPrivilege Enum:** Represents roles: Guest, Developer, Secretary, and DBA using [Flags] for privilege combinations.
- **Employee Class:** Holds ID, Name, SecurityPrivilege, Salary, HiringDate, and Gender. Uses properties, custom constructor, and overrides ToString() with currency salary formatting.
- **Main Program:** Creates an array of 3 employees (DBA, Guest, and Security Officer with full permissions), accepts user input safely without runtime errors, and prints details.

1. Gender Enum

We restrict the gender to Male or Female using an enum as requested.

```
using System;

namespace Assignment06
{
    public enum Gender
    {
        M = 1, // Male
        F = 2  // Female
    }
}
```

2. Security Privilege Enum (Flags)

We use [Flags] attribute so we can combine privileges for roles like Security Officer (who gets Guest, Developer, Secretary, and DBA all together).

```
using System;

namespace Assignment06
{
    [Flags]
    public enum SecurityPrivilege : byte
    {
        None = 0,
```

```

        Guest = 1,          // 0001
        Developer = 2,      // 0010
        Secretary = 4,      // 0100
        DBA = 8             // 1000
    }
}

```

3. HiringDate Class

This class represents the date an employee was hired. It includes properties for Day, Month, and Year with validation in setters.

```

using System;

namespace Assignment06
{
    public class HiringDate
    {
        private int day;
        private int month;
        private int year;

        // Default Constructor
        public HiringDate()
        {
            day = 1;
            month = 1;
            year = 2020;
        }

        // Parameterized Constructor
        public HiringDate(int day, int month, int year)
        {
            Day = day;
            Month = month;
            Year = year;
        }

        // Properties with basic encapsulation & validation
        public int Day
        {
            get { return day; }
            set
            {
                if (value >= 1 && value <= 31)
                    day = value;
                else
                    day = 1; // Default fallback if invalid
            }
        }

        public int Month
        {
            get { return month; }

```

```

        set
        {
            if (value >= 1 && value <= 12)
                month = value;
            else
                month = 1;
        }
    }

    public int Year
    {
        get { return year; }
        set
        {
            if (value >= 1900 && value <= DateTime.Now.Year)
                year = value;
            else
                year = 2020;
        }
    }

    // String representation format: DD/MM/YYYY
    public override string ToString()
    {
        return $"{day:D2}/{month:D2}/{year}";
    }
}

```

4. Employee Class

The main class encapsulating all fields, properties, constructors, and overriding `ToString()` to format salary as currency using `String.Format`.

```

using System;

namespace Assignment06
{
    public class Employee
    {
        // Private Fields
        private int id;
        private string name;
        private SecurityPrivilege securityLevel;
        private decimal salary;
        private HiringDate hireDate;
        private Gender gender;

        // Constructors
        public Employee()
        {
            id = 0;
            name = "Unknown";
            securityLevel = SecurityPrivilege.Guest;
        }
    }
}

```

```

        salary = 3000m;
        hireDate = new HiringDate();
        gender = Gender.M;
    }

    public Employee(int id, string name, SecurityPrivilege securityLevel, decimal
salary, HiringDate hireDate, Gender gender)
    {
        Id = id;
        Name = name;
        SecurityLevel = securityLevel;
        Salary = salary;
        HireDate = hireDate ?? new HiringDate();
        Gender = gender;
    }

    // Properties
    public int Id
    {
        get { return id; }
        set { id = value > 0 ? value : 1; }
    }

    public string Name
    {
        get { return name; }
        set { name = string.IsNullOrEmpty(value) ? "No Name" : value; }
    }

    public SecurityPrivilege SecurityLevel
    {
        get { return securityLevel; }
        set { securityLevel = value; }
    }

    public decimal Salary
    {
        get { return salary; }
        set { salary = value >= 0 ? value : 0; }
    }

    public HiringDate HireDate
    {
        get { return hireDate; }
        set { hireDate = value ?? new HiringDate(); }
    }

    public Gender Gender
    {
        get { return gender; }
        set { gender = value; }
    }

    // Overriding ToString method with String.Format for currency
    public override string ToString()

```

```

        {
            string formattedSalary = String.Format("{0:C}", salary);
            return $"ID: {id} | Name: {name} | Gender: {gender} | Security:
{securityLevel} | Salary: {formattedSalary} | Hire Date: {hireDate}";
        }
    }
}

```

5. Main Program with Error Handling

In the main program, we create an array of 3 employees (DBA, Guest, and Security Officer with full permissions). We handle all user inputs safely using `int.TryParse` and `Enum.TryParse` to prevent any runtime exceptions.

```

using System;

namespace Assignment06
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("=====");
            Console.WriteLine("            EMPLOYEE MANAGEMENT SYSTEM - ASSIGNMENT 6 ");
            Console.WriteLine("=====");

            // Array of size 3 as requested
            Employee[] EmpArr = new Employee[3];

            // Define roles for the 3 required employees
            // 1. DBA
            // 2. Guest
            // 3. Security Officer (Full permissions = Guest | Developer | Secretary
| DBA)
            SecurityPrivilege[] predefinedRoles = new SecurityPrivilege[]
            {
                SecurityPrivilege.DBA,
                SecurityPrivilege.Guest,
                SecurityPrivilege.Guest | SecurityPrivilege.Developer |
SecurityPrivilege.Secretary | SecurityPrivilege.DBA
            };

            string[] roleTitles = new string[] { "DBA", "Guest", "Security Officer
(Full Permissions)" };

            for (int i = 0; i < EmpArr.Length; i++)
            {
                Console.WriteLine($"--- Enter Details for Employee #{i + 1}
[{roleTitles[i]}] ---");

                // 1. Read ID safely
                int id;
            }
        }
    }
}

```

```

do
{
    Console.WriteLine("Enter Employee ID: ");
} while (!int.TryParse(Console.ReadLine(), out id) || id <= 0);

// 2. Read Name safely
string name;
do
{
    Console.WriteLine("Enter Employee Name: ");
    name = Console.ReadLine();
} while (string.IsNullOrEmpty(name));

// 3. Read Gender safely (M / F)
Gender gender;
while (true)
{
    Console.WriteLine("Enter Gender (M/F): ");
    string inputGender = Console.ReadLine()?.Trim().ToUpper();
    if (inputGender == "M")
    {
        gender = Gender.M;
        break;
    }
    else if (inputGender == "F")
    {
        gender = Gender.F;
        break;
    }
    Console.WriteLine("Invalid input! Please enter 'M' for Male or
'F' for Female.");
}

// 4. Read Salary safely
decimal salary;
do
{
    Console.WriteLine("Enter Salary: ");
} while (!decimal.TryParse(Console.ReadLine(), out salary) || salary
< 0);

// 5. Read Hire Date safely
Console.WriteLine("Enter Hiring Date:");
int day, month, year;

do
{
    Console.WriteLine(" Day (1-31): ");
} while (!int.TryParse(Console.ReadLine(), out day) || day < 1 || day
> 31);

do
{
    Console.WriteLine(" Month (1-12): ");
} while (!int.TryParse(Console.ReadLine(), out month) || month < 1 ||

```

```

month > 12);

        do
        {
            Console.Write("  Year (1900-2026): ");
        } while (!int.TryParse(Console.ReadLine(), out year) || year < 1900
|| year > DateTime.Now.Year);

        HiringDate hireDate = new HiringDate(day, month, year);

        // Assign security privilege according to assignment requirement
        SecurityPrivilege securityLevel = predefinedRoles[i];

        // Create and store Employee instance
        EmpArr[i] = new Employee(id, name, securityLevel, salary, hireDate,
gender);

        Console.WriteLine("Employee added successfully!
");

    }

    // Display all employees
    Console.WriteLine("
=====");
    Console.WriteLine("                      EMPLOYEE RECORDS LIST                      ");
    Console.WriteLine("=====");

    for (int i = 0; i < EmpArr.Length; i++)
    {
        Console.WriteLine($"
[Employee {i + 1}]");
        Console.WriteLine(EmpArr[i].ToString());
    }

    Console.WriteLine("
Press Any Key to Exit...");
    Console.ReadKey();
}
}
}

```

Sample Output (Console Screen)

```

=====
EMPLOYEE MANAGEMENT SYSTEM - ASSIGNMENT 6
=====

--- Enter Details for Employee #1 [DBA] ---
Enter Employee ID: 101
Enter Employee Name: Ahmed Nofal
Enter Gender (M/F): M
Enter Salary: 12500
Enter Hiring Date:
  Day (1-31): 15
  Month (1-12): 6

```

```
Year (1900-2026): 2022
Employee added successfully!
```

```
--- Enter Details for Employee #2 [Guest] ---
```

```
Enter Employee ID: 102
Enter Employee Name: Mona Ali
Enter Gender (M/F): F
Enter Salary: 4500
Enter Hiring Date:
  Day (1-31): 10
  Month (1-12): 10
  Year (1900-2026): 2024
Employee added successfully!
```

```
--- Enter Details for Employee #3 [Security Officer (Full Permissions)] ---
```

```
Enter Employee ID: 103
Enter Employee Name: Mahmoud Hassan
Enter Gender (M/F): M
Enter Salary: 18000
Enter Hiring Date:
  Day (1-31): 1
  Month (1-12): 1
  Year (1900-2026): 2020
Employee added successfully!
```

```
=====
EMPLOYEE RECORDS LIST
=====
```

```
[Employee 1]
ID: 101 | Name: Ahmed Nofal | Gender: M | Security: DBA | Salary: $12,500.00 | Hire Date:
15/06/2022
```

```
[Employee 2]
ID: 102 | Name: Mona Ali | Gender: F | Security: Guest | Salary: $4,500.00 | Hire Date:
10/10/2024
```

```
[Employee 3]
ID: 103 | Name: Mahmoud Hassan | Gender: M | Security: Guest, Developer, Secretary, DBA |
Salary: $18,000.00 | Hire Date: 01/01/2020
```

Note on Runtime Errors Prevention: All inputs use TryParse and input validation loops. If a user inputs text instead of numbers or invalid dates, the program asks them to enter the value again smoothly without crashing.